Maven — Interview Revision Notes

> **Goal:** Understand why Maven exists and how it helps Java projects manage **project structure, dependencies, builds, packaging, repositories, and sharing artifacts**.

---

1. Basic Journey of Java Code

At the core of Java:

```
Hello.java
    |
    | javac
    v
Hello.class
    |
    | JVM
    v
Output
```

- `.java` → source code written by the developer.
- `.class` → compiled Java bytecode.
- JVM runs the `.class` bytecode.

Interview recall: Java source code is not directly executed by the JVM. It is first compiled into bytecode.

---

2. What Happens in a Real Java Project?

A real project contains many Java files:

```
User.java
Order.java
Payment.java
EmailService.java
InvoiceGenerator.java
Main.java
```

After compilation:

```
User.class
Order.class
Payment.class
EmailService.class
InvoiceGenerator.class
Main.class
```

Sharing many individual `.class` files is inconvenient because:

- Files can get misplaced.
- Package/folder structure can break.
- Files can be missed.
- Resource files may not be included.
- Running the project becomes confusing.

Solution → JAR

Java provides a standard way to bundle compiled code and related files: **JAR (Java Archive)**.

---

3. What is a JAR File?

JAR = Java Archive

A JAR is a packaged bundle for Java projects. It can be thought of as a ZIP-like archive designed for Java.

A JAR can contain:

- `.class` files
- packages/folders

- images
- properties files
- configuration files
- metadata

```
Java Project
     |
     | compile + package
     v
+----------------------+
|      app.jar         |
|----------------------|
| .class files         |
| packages             |
| resources            |
| configuration        |
| metadata             |
+----------------------+
```

Why is JAR important?

Reason 1 — Share Java code

Instead of sending many `.class` files:

```
Calculator classes
       |
       v
calculator.jar
       |
       v
Another Java project
```

Another developer can add the JAR to their project and use the compiled code.

Reason 2 — Use external libraries

When we say:

- Spring Core
- MySQL Connector
- Hibernate

- Jackson

we are practically using their compiled Java code, commonly distributed as JAR files.

Interview recall: A Java library is often compiled Java code packaged as a JAR.

---

4. Library vs Application

Library	Application
Code created for other developers to use	Code created for end users
Usually does not run independently	Can run as a complete program
Provides reusable functionality	Provides a complete application
Example: calculator library	Example: calculator application

Example:

```
calculator.jar
     |
     +-- reusable functions only
          -> Library JAR

calculator.jar
     |
     +-- complete runnable program
          -> Application JAR
```

---

5. Normal Library JAR vs Spring Boot Executable JAR

A normal library JAR usually contains its own compiled classes, while dependency information is managed separately.

A Spring Boot application is commonly packaged as an **executable JAR**.

```
Normal Library JAR
    |
    +-- Your compiled classes
    +-- dependency information
    |
    └-- dependencies usually supplied separately

Spring Boot Executable JAR
    |
    +-- application code
    +-- required dependencies
    |
    └-- runnable as one executable JAR
```

A Spring Boot executable JAR is often called a:

- **Fat JAR**
- **Executable JAR**

> **Important:** Do not assume every JAR contains all dependencies. The source specifically distinguishes normal library JARs from Spring Boot executable JARs.

---

6. External Libraries and Classpath

Suppose our Java code uses an external JAR:

```
Your Java Code
      +
External JAR
      |
      v
  Compile
      |
      v
     JVM
```

Java needs to know where required classes are located.

This is handled using the **classpath**.

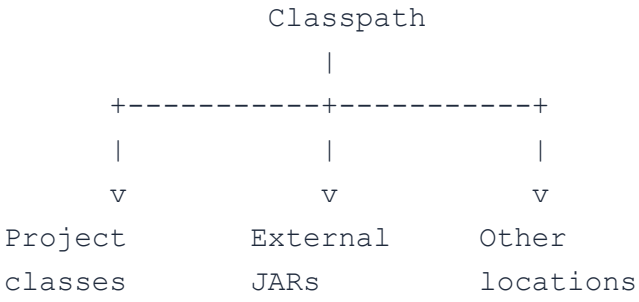
---

7. What is Classpath?

Classpath = locations where Java searches for required classes.

Java can search:

- Project classes
- External JAR files
- Other configured class locations



Interview definition

> The classpath tells Java where to search for required classes.

---

8. Why Do We Need Maven?

If a project needs one external JAR, we could manually download it.

But real projects may need many libraries.

Manual management creates problems:

```
Many JARs
   |
   +-- Where to download?
   +-- Which version?
   +-- Dependencies of dependencies?
   +-- Different versions between teammates?
   +-- Missing JAR?
   +-- Compatibility problems?
```

Maven's solution

Instead of manually downloading and managing JARs:

```
Developer
   |
   | declares required dependency
   v
pom.xml
   |
   v
Maven
   |
   +--> finds dependency
   +--> downloads dependency
   +--> manages versions
   +--> manages transitive dependencies
   +--> builds project
```

---

9. What is Maven?

Maven is a project management and build automation tool for Java projects.

Maven is **independent of Spring**.

It can be used with:

- Core Java
- Spring
- Spring Boot

- Hibernate
- Other Java projects

Maven helps with

1. Standard project structure
2. Dependency management
3. Compilation
4. Testing
5. Packaging
6. Build-related tasks
7. Plugin management
8. Project configuration

Interview answer:

> Maven is a Java project management and build automation tool that standardizes project structure, manages dependencies, executes builds and tests, and packages artifacts.

---

10. What Maven Does Practically

Maven responsibility	What it does
Project structure	Provides standard structure
Compile	Compiles Java source
Test	Runs tests
Package	Creates JAR/WAR
Dependency management	Downloads and manages libraries
Version management	Manages dependency versions
Plugins	Performs build-related tasks

---

11. Maven and IDEs

Maven can be used from the command line:

```
mvn clean install
```

Modern IDEs also support Maven:

- IntelliJ IDEA
- Eclipse
- VS Code

Why Maven is useful across IDEs

Each IDE may have its own project handling, but Maven provides a **standard project structure** that works across tools.

---

12. Convention Over Configuration

Maven follows:

> **Convention Over Configuration**

Meaning: Maven already assumes a standard project structure.

For example:

```
src/main/java   -> main Java source
src/test/java   -> test Java source
```

Because Maven knows these conventions, we do not have to manually configure every location.

Key idea

```
Follow Maven conventions
          |
          v
Maven automatically knows
where source/test/resources/output belong
```

---

13. Standard Maven Project Structure

```
my-maven-project/
│
├── pom.xml
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   └── resources/
│   │
│   └── test/
│       ├── java/
│       └── resources/
│
└── target/
```

`pom.xml`

Contains information about the Maven project:

- Project name
- Project version
- Dependencies
- Plugins
- Build configuration
- Parent configuration

`src/main/java`

Main Java source code:

- Controller
- Service
- Repository
- Utility
- Main application class

`src/main/resources`

Non-Java resources needed by the application:

- `application.properties`
- `application.yml`
- Configuration files
- Static files
- Templates

`src/test/java`

Test code:

- Unit tests
- Integration tests

`src/test/resources`

Resources required only during testing:

- Test configuration
- Test data

`target`

Generated by Maven during builds.

It can contain:

- `.class` files

- Test reports
- Generated sources
- Final JAR/WAR
- Temporary build files

> **Important:** `target` is generated output, not source code. Maven can recreate it.

---

14. The Most Important Maven File — `pom.xml`

POM = Project Object Model

`pom.xml` is the **heart of a Maven project**.

It tells Maven:

- What the project is
- Project version
- Output type
- External libraries
- Plugins
- Parent configuration

---

15. Basic `pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xsi="http://www.w3.org/2001/XMLSchema-instance"
         schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.coderarmy</groupId>
    <artifactId>calculator-app</artifactId>
    <version>1.0.0</version>

</project>
```

The XML namespace lines define the XML namespace/schema information. For beginner-level understanding, the important POM elements are the Maven configuration tags themselves.

---

16. `<modelVersion>`

```
<modelVersion>4.0.0</modelVersion>
```

It tells Maven which POM model version the file follows.

Most Maven projects use:

```
4.0.0
```

---

17. Maven Coordinates

Three important tags:

```
<groupId>com.coderarmy</groupId>
<artifactId>calculator-app</artifactId>
<version>1.0.0</version>
```

Together they are called **Maven coordinates**.

```
groupId + artifactId + version
                |
                v
      Identify an artifact
```

`groupId`

Usually represents the organization, company, or domain owning the project.

Example:

```
<groupId>com.coderarmy</groupId>
```

`artifactId`

Name of the project/module.

```
<artifactId>calculator-app</artifactId>
```

`version`

Version of the project/artifact.

```
<version>1.0.0</version>
```

Interview recall

> Maven coordinates = `groupId + artifactId + version`

---

18. SNAPSHOT Version

Example:

```
<version>1.0.0-SNAPSHOT</version>
```

`SNAPSHOT` indicates a project version that is still under development and is not considered a final stable release.

```
1.0.0
   -> stable release

1.0.0-SNAPSHOT
   -> development version
```

---

19. `<packaging>`

The `<packaging>` tag tells Maven what type of output should be created.

Example:

```
<packaging>jar</packaging>
```

Common packaging types:

Packaging	Meaning
`jar`	Creates JAR
`war`	Creates WAR
`pom`	Parent/aggregator project

If packaging is not specified, Maven defaults to:

```
jar
```

20. `<properties>`

`<properties>` defines reusable values in the POM.

Example:

```
<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

This tells Maven:

- Use Java 17 source code.
- Compile for Java 17.
- Use UTF-8 encoding.

Why properties?

They make the POM cleaner because values can be reused.

---

21. `<dependencies>`

The `<dependencies>` section contains external libraries required by the project.

Example:

```
<dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.x.x</version>
    </dependency>
</dependencies>
```

Each dependency is identified using:

```
groupId + artifactId + version
```

Maven uses these coordinates to find the required artifact in a repository.

---

22. Hibernate Dependency Example

```
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.x.x</version>
</dependency>
```

This tells Maven that the project needs Hibernate Core.

Maven downloads the Hibernate JAR and manages its required supporting dependencies.

---

23. Spring Boot Starter Dependency

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Important point

A starter dependency does **not** mean only one JAR.

It usually brings a **group of related dependencies** needed for a feature.

```
spring-boot-starter-web
          |
          +-- related web dependencies
          +-- supporting dependencies
          +-- other required dependencies
```

---

24. Transitive Dependencies

Suppose:

```
Your Project
     |
     v
     A
     |
     v
     B
     |
     v
     C
```

If:

```
A depends on B
B depends on C
```

and you add A, Maven can automatically bring B and C.

These are called **transitive dependencies**.

Definition

> Transitive dependencies are dependencies of your dependencies.

```
Direct dependency
        A
        |
        +--> B
                |
                +--> C

B and C = transitive dependencies
```

Why important?

Without Maven, we might have to manually find every required JAR.

---

25. Maven Plugins

Dependency vs Plugin

Dependency:

> A library used by our application code.

Plugin:

> A tool used by Maven during the build process.

Plugins can:

- Compile code
- Run tests
- Create JARs
- Create reports
- Run Spring Boot applications
- Package applications

Example:

Spring Boot Maven Plugin

It helps package and run Spring Boot applications.

Interview comparison

```
Dependency
    |
    v
Used by application code

Plugin
    |
    v
Used by Maven during build
```

---

26. Maven Repositories

Maven downloads dependencies from **Maven repositories**.

A repository is a storage place for Maven artifacts.

Artifacts can include:

- JAR
- WAR
- POM
- Plugins
- Other build artifacts

---

27. Types of Maven Repositories

At beginner level, remember:

```
1. Local Repository
2. Maven Central Repository
3. Remote Repository
```

28. Local Repository

The local repository is a folder on your own computer.

Windows

```
C:\Users\<your-username>\.m2\repository
```

Mac/Linux

```
~/.m2/repository
```

It has two major jobs:

1. Cache downloaded dependencies.
2. Store locally installed artifacts.

First vs Later Build

```
First build
   |
   +--> Download dependencies
   +--> Store in .m2
   |
   v
Usually slower

Later build
   |
   +--> Reuse cached dependencies
   |
   v
Usually faster
```

29. What Happens During `mvn install`?

Command:

```
mvn install
```

Maven builds the project and stores the final artifact in the local repository.

Example:

```
~/.m2/repository/com/example/MavenDemo/1.0/
```

This allows another local Maven project on the same machine to use that artifact as a dependency.

---

30. Maven Central Repository

Maven Central is a large public repository containing many open-source Java libraries.

Examples commonly available:

- Spring
- Hibernate
- Jackson
- JUnit
- MySQL Connector

When a public dependency is not already local, Maven usually obtains it from Maven Central.

---

31. Remote Repository

A remote repository is any Maven repository that is **not on your local machine**.

It can be hosted on:

- Internet
- Company network
- Private repository server

Examples:

- Maven Central
- Company Nexus
- Company Artifactory
- GitHub Packages

---

32. Why Companies Use Private Maven Repositories

Not every company library is public.

A company may create internal libraries that should be available only inside the organization.

Common private repository solutions include:

- Nexus
- Artifactory
- GitHub Packages

Example configuration:

```
<repositories>
    <repository>
        <id>company-repo</id>
        <url>https://repo.company.com/maven2</url>
    </repository>
</repositories>
```

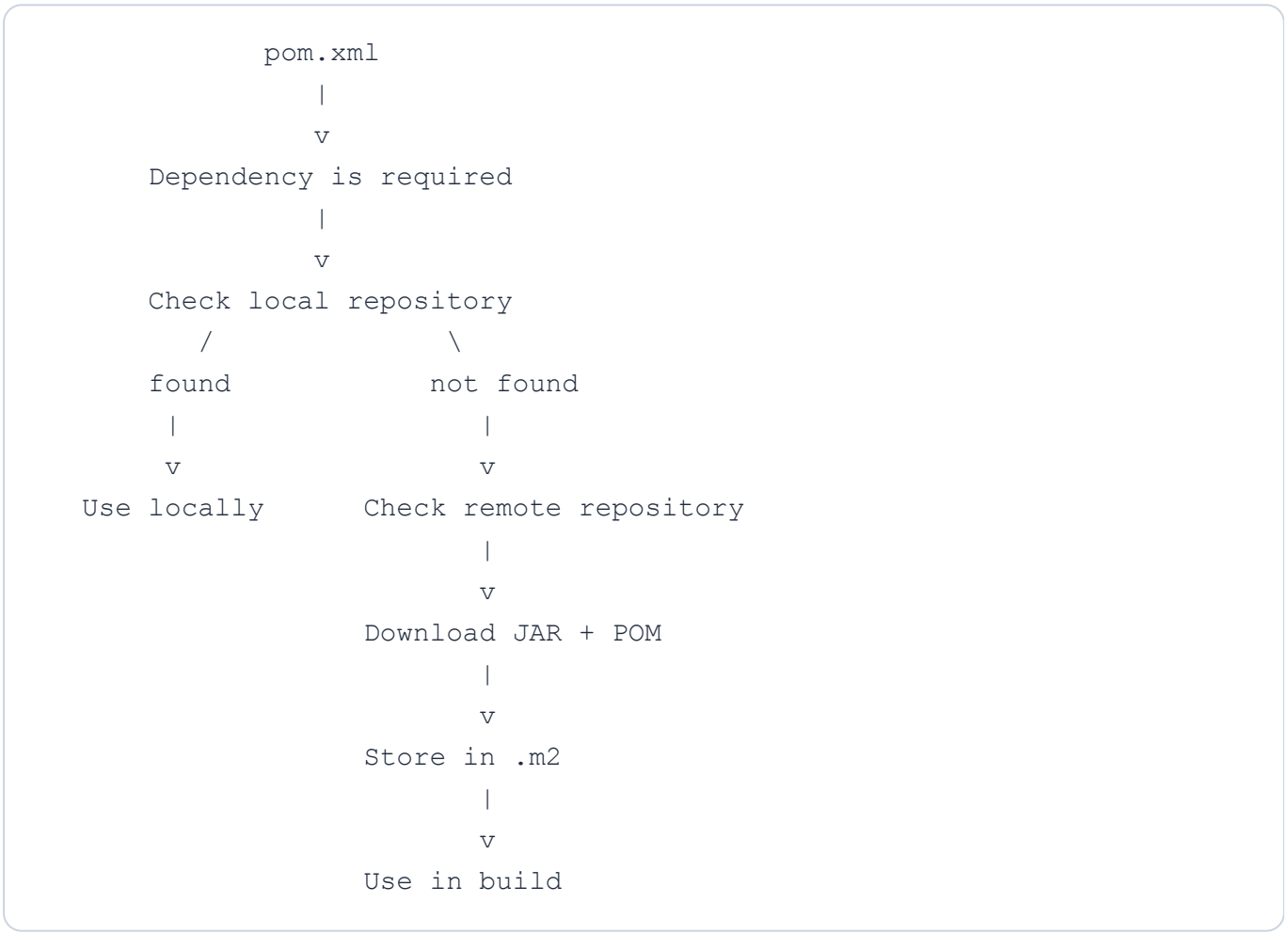
---

33. How Maven Searches for a Dependency

Suppose `pom.xml` contains:

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
</dependency>
```

Maven roughly follows:



Key point

The first build can be slower because Maven downloads dependencies.

Later builds can reuse cached dependencies.

---

34. `settings.xml`

`settings.xml` is Maven's **user-level configuration file**.

Common location:

```
~/.m2/settings.xml
```

It can contain:

- Repository credentials
- Proxy settings
- Mirror configuration
- Custom local repository location

Important comparison

```
pom.xml
    -> project-level configuration

settings.xml
    -> user/machine-level Maven configuration
```

---

35. Why Deleting `.m2/repository` Sometimes Fixes Dependency Errors

Sometimes local Maven dependency files become corrupted or incomplete.

Possible reasons:

- Download failed halfway.
- Internet disconnected during download.
- Partial JAR was saved.
- Metadata became inconsistent.

Deleting the affected dependency folder forces Maven to download it again.

```
Corrupted dependency in .m2
          |
          v
Delete affected dependency
          |
          v
Run Maven build
          |
          v
Maven downloads it again
```

Important

Deleting the **entire** `.m2/repository` means Maven has to download **all dependencies again**.

Prefer deleting only the problematic dependency when possible.

---

36. Maven Lifecycle

A Java project has a build journey:

```
Check project
     |
     v
Compile code
     |
     v
Run tests
     |
     v
Create JAR/WAR
     |
     v
Install / Deploy artifact
```

Maven calls this ordered build journey a **lifecycle**.

Definition

> A Maven lifecycle is a standard sequence of steps Maven follows to build and manage a project.

---

37. MOST IMPORTANT Maven Lifecycle Rule

> **When we run a Maven phase, Maven automatically runs all earlier phases of that lifecycle before it.**

Example:

```
mvn package
```

Conceptually:

```
validate
   ↓
compile
   ↓
test
   ↓
package
```

Example:

```
mvn install
```

Conceptually:

```
validate
   ↓
compile
   ↓
test
   ↓
package
   ↓
verify
   ↓
install
```

Interview trap

`mvn package` does **not** mean "only package".

It runs the required earlier phases first.

---

38. Three Main Maven Lifecycles

Maven has three main lifecycles:

```
1. Clean
2. Default
3. Site
```

Lifecycle	Purpose
Clean	Cleans old build output
Default	Builds, tests, packages, installs and deploys
Site	Generates documentation and reports

---

39. Clean Lifecycle

Used to remove old build output.

Command:

```
mvn clean
```

Usually deletes:

```
target/
```

Why use it?

When we want a fresh build.

```
Old target
     |
     | mvn clean
     v
Removed
     |
     | next build
     v
Fresh target
```

---

40. Default Lifecycle

The **Default lifecycle** is the main build lifecycle.

Important phases:

```
validate
   ↓
compile
   ↓
test
   ↓
package
   ↓
verify
   ↓
install
   ↓
deploy
```

---

41. Important Default Lifecycle Phases

Phase	Meaning
`validate`	Checks whether project is valid
`compile`	Compiles `.java` → `.class`
`test`	Runs test cases
`package`	Creates JAR/WAR
`verify`	Performs additional checks
`install`	Stores artifact in local `.m2`
`deploy`	Uploads artifact to remote repository

---

42. `validate` Phase

Checks whether the project is structurally valid.

Examples:

- Is `pom.xml` present?
- Is POM readable?
- Are required project details available?
- Is the project structure valid enough to continue?

Command:

```
mvn validate
```

---

43. `compile` Phase

Compiles main Java source code.

Command:

```
mvn compile
```

Flow:

```
src/main/java
      |
      | compile
      v
target/classes
      |
      +-- .class files
```

---

44. `test` Phase

Runs test cases.

Command:

```
mvn test
```

Tests are located under:

```
src/test/java
```

Maven compiles and runs the tests.

Purpose:

> Verify that the code works as expected before packaging.

---

45. `package` Phase

Creates the final packaged output.

Command:

```
mvn package
```

For a normal Java project, this commonly creates a JAR:

```
target/MavenDemo-1.0.jar
```

For a web application, it may create a WAR depending on packaging configuration.

Flow:

```
Compiled code + resources
          |
          | package
          v
       JAR/WAR
```

---

46. `verify` Phase

Checks whether the packaged output is valid.

Command:

```
mvn verify
```

In larger projects it can include:

- Integration tests
- Code quality checks
- Coverage checks
- Additional validation

Interview recall

> `verify` performs additional checks before installing or deploying the artifact.

---

47. `install` Phase

Builds the project and stores the final artifact in the **local Maven repository**.

Command:

```
mvn install
```

Artifact goes into:

```
~/.m2/repository
```

Useful when another local Maven project needs this project as a dependency.

Flow:

```
Project
   |
   | mvn install
   v
Build + Package
   |
   v
Local .m2 repository
```

---

48. `deploy` Phase

Builds the project and uploads the final artifact to a **remote Maven repository**.

Command:

```
mvn deploy
```

Examples of remote repositories:

- Company Nexus
- Company Artifactory
- GitHub Packages
- Other remote Maven repositories

Important: GitHub vs Maven Repository

```
GitHub
  -> usually source code

Maven remote repository
  -> built artifacts such as JARs
```

So:

> `mvn deploy` is not the same as pushing source code to GitHub.

---

49. Site Lifecycle

The Site lifecycle generates project documentation and reports.

Command:

```
mvn site
```

Output usually goes into:

```
target/site
```

It is separate from the normal build flow.

---

50. What Happens with `mvn clean install`?

Command:

```
mvn clean install
```

This combines:

```
clean lifecycle
      +
default lifecycle up to install
```

Step 1 — Clean

Old build output is removed, usually:

```
target/
```

Step 2 — Default lifecycle

```
validate
   ↓
compile
   ↓
test
   ↓
package
   ↓
verify
   ↓
install
```

Complete meaning

```
mvn clean install
        |
        +--> clean old output
        |
        +--> build project
        |
        +--> run tests
        |
        +--> package project
        |
        +--> install artifact into .m2
```

Interview one-liner:

> `mvn clean install` removes old build output and then executes the default lifecycle up to `install`, storing the built artifact in the local Maven repository.

---

51. Maven Archetypes

A **Maven archetype** is a project template.

It provides:

- Ready-made structure
- Basic configuration
- Boilerplate files

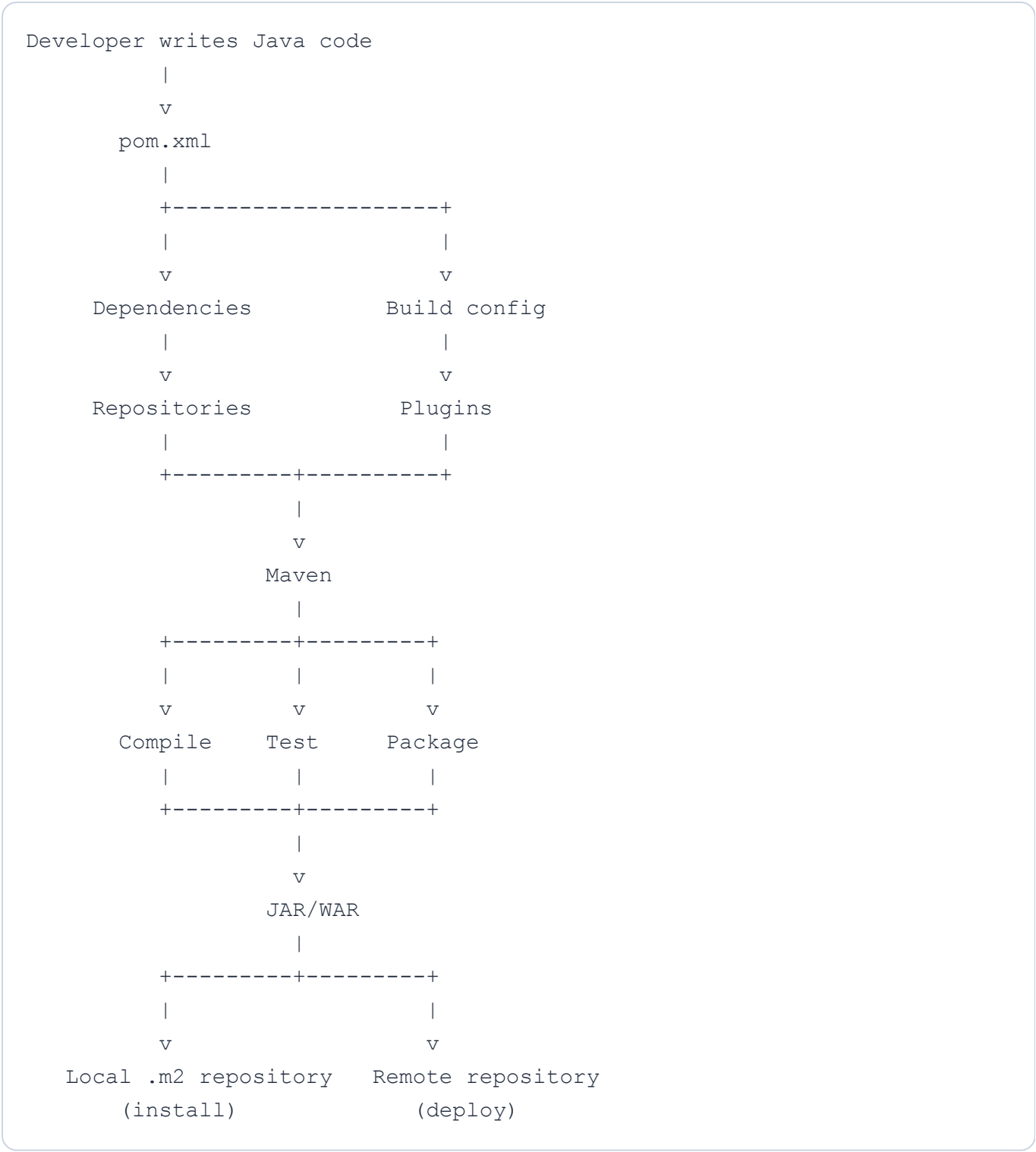
Conceptually:

```
Archetype
    |
    v
Project template
    |
    +-- folder structure
    +-- configuration
    +-- boilerplate
    |
    v
New Maven project
```

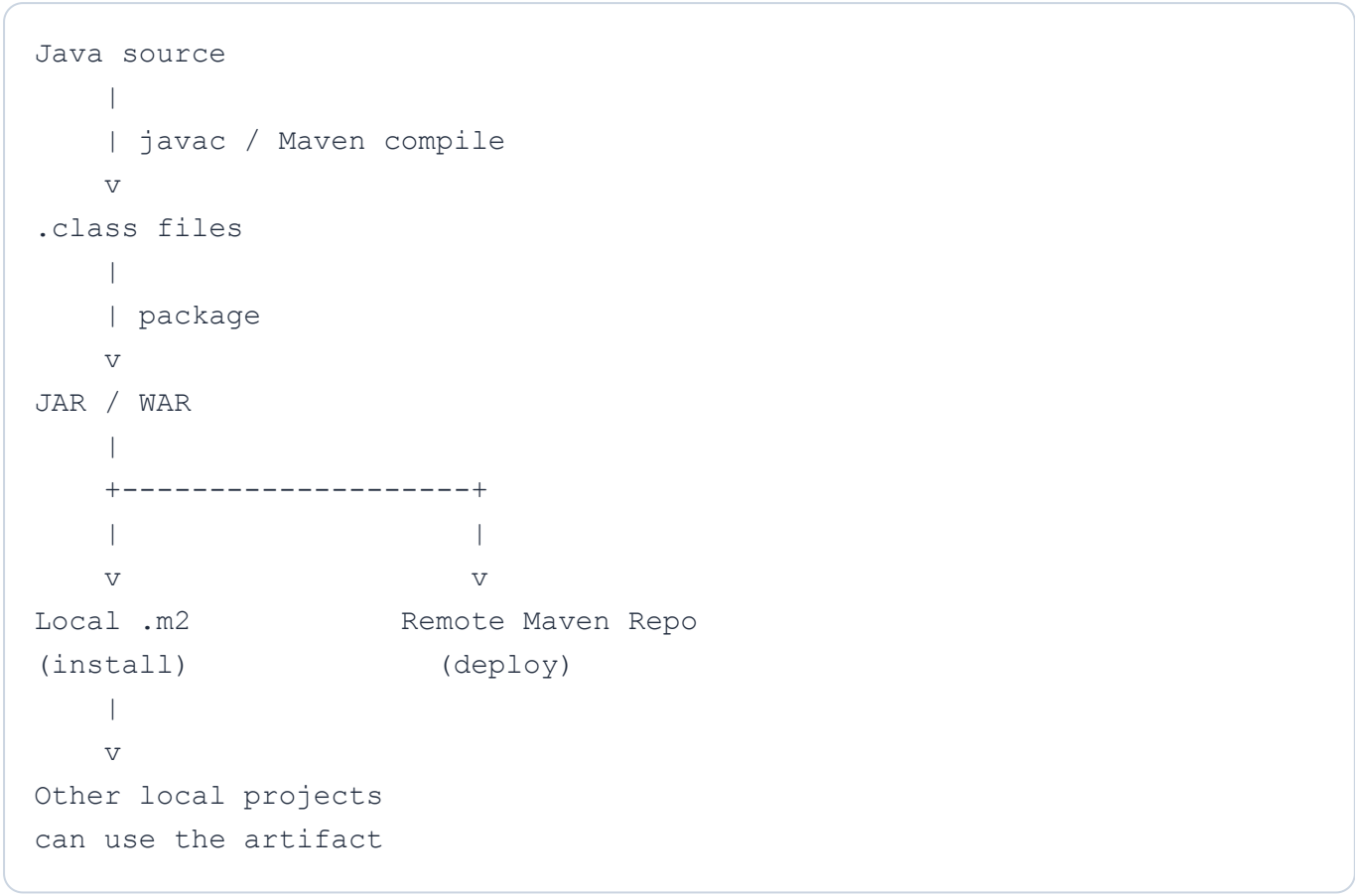
Modern Spring Boot projects are commonly created using **Spring Initializr**, but Maven archetypes remain an important Maven concept.

---

52. Complete Maven Flow — Remember This



53. End-to-End Mental Model



54. Maven: Dependency Flow

```
pom.xml
   |
   | groupId + artifactId + version
   v
Maven
   |
   v
Check local .m2
   |
   +---- Found ----> Use dependency
   |
   +---- Not found
            |
            v
     Remote repository
            |
            v
     Download JAR + POM
            |
            v
       Store in .m2
            |
            v
       Use in project
```

---

55. Maven: Dependency vs Plugin

Dependency	Plugin
Used by application	Used by Maven
Provides library functionality	Performs build tasks
Example: Hibernate	Example: compiler plugin
Declared in `<dependencies>`	Build plugins are configured under build/plugin configuration

Easy memory trick:

> **Dependency → Application needs it**
> **Plugin → Maven needs it**

---

56. Maven: `install` vs `deploy`

`install`	`deploy`
Stores artifact locally	Uploads artifact remotely
Uses `.m2/repository`	Uses remote Maven repository
Useful for other local projects	Useful for sharing artifacts with other environments/projects

```
mvn install
      |
      v
Local .m2

mvn deploy
      |
      v
Remote Maven repository
```

---

57. Maven: GitHub vs Maven Repository

Do not confuse source-code hosting with artifact repositories.

```
Source code
    |
    v
GitHub repository

Built JAR/WAR
    |
    v
Maven repository
```

A Maven remote repository can be used so other projects can declare the artifact as a dependency.

---

58. Maven: `pom.xml` vs `settings.xml`

`pom.xml`	`settings.xml`
Project-level	User/machine-level
Describes the project	Configures Maven for the user/machine
Dependencies	Credentials
Plugins	Proxy
Build configuration	Mirrors
Project coordinates	Custom local repository

Interview recall:

> `pom.xml` describes **the project**; `settings.xml` configures **Maven for the user/machine**.

---

59. Maven: `src` vs `target`

```
src/
  |
  +-- source code
  +-- tests
  +-- resources

target/
  |
  +-- compiled classes
  +-- reports
  +-- generated files
  +-- JAR/WAR
  +-- temporary build output
```

Important: `target` is generated output and can be recreated.

---

60. Interview Quick Recall

Q1. What is Maven?

> Maven is a Java project management and build automation tool used for standard project structure, dependency management, compilation, testing, packaging, and build automation.

Q2. Why do we need Maven?

> Manually managing many JARs, versions, dependencies, and builds is difficult. Maven standardizes and automates these tasks.

Q3. What is JAR?

> JAR stands for Java Archive and packages compiled Java classes and related resources into one archive.

Q4. What is classpath?

> Classpath tells Java where to search for required classes.

Q5. What is POM?

> POM stands for Project Object Model. `pom.xml` is the central configuration file of a Maven project.

Q6. What are Maven coordinates?

> `groupId + artifactId + version`.

Q7. What is a transitive dependency?

> A dependency required by another dependency.

Q8. Dependency vs plugin?

> Dependency is used by application code; plugin is used by Maven during the build.

Q9. Where are downloaded dependencies stored?

> In the local Maven repository, commonly `.m2/repository`.

Q10. What happens in `mvn install`?

> Maven builds the project and stores the final artifact in the local Maven repository.

Q11. What happens in `mvn deploy`?

> Maven uploads the built artifact to a remote Maven repository.

Q12. What does `mvn clean` do?

> It removes old build output, usually the `target` directory.

Q13. What does `mvn package` do?

> It executes earlier required phases and packages the application as a JAR/WAR according to the project configuration.

Q14. What is the Maven lifecycle?

> A standard sequence of phases Maven follows to build and manage a project.

Q15. What are the three Maven lifecycles?

> Clean, Default, and Site.

Q16. What does `mvn clean install` mean?

> Clean old build output, execute the default lifecycle up to install, and store the artifact in the local repository.

---

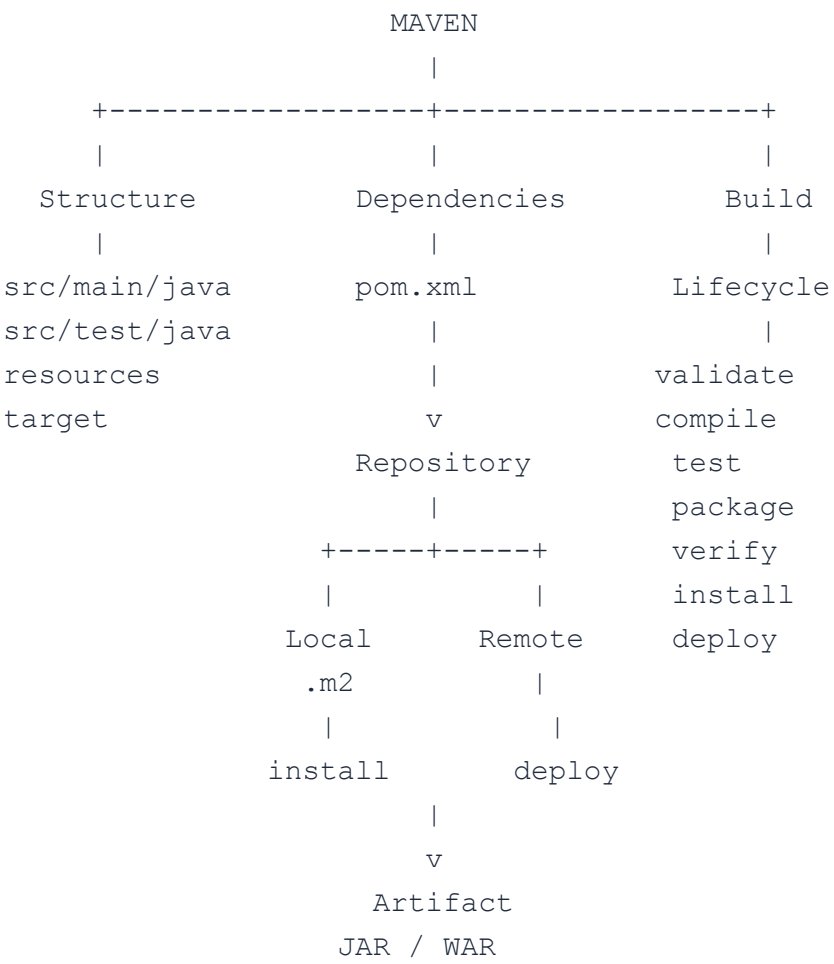
61. Commands to Remember

```
mvn validate
mvn compile
mvn test
mvn package
mvn verify
mvn install
mvn deploy
mvn clean
mvn clean install
mvn site
```

Most important build flow

```
validate
   ↓
compile
   ↓
test
   ↓
package
   ↓
verify
   ↓
install
   ↓
deploy
```

62. Final Interview Mental Map



---

63. One-Line Takeaway

> **Maven exists because real Java projects need a standard way to manage project structure, dependencies, builds, packaging, and sharing of artifacts.**